

**LAPORAN TUGAS BESAR & UAS PBO
PENGUNAAN REST API PADA PENGUSULAN JUDUL BUKU
PERPUSTAKAAN UMUM INDRAMAYU**



**Disusun oleh;
Muhamamd Rafli (2303099)**

**POLITEKNIK NEGERI INDRAMAYU
Jl.Raya Lohbener lama No.08 Indramayu
45252 Fax/ Tlp.(0231) 5746464**

A. DESKRIPSI

REST API yang dikembangkan untuk bagian dari objek buku dirancang sebagai solusi untuk memudahkan proses mencari entitas atau pengelolaan sesuai dengan kebutuhan pengguna. Fungsi REST API dari koleksi buku perpustakaan nasional atau internasional adalah untuk menyediakan akses programatik ke data katalog buku yang dimiliki oleh perpustakaan tersebut. Dengan REST API, pengembang dapat mengambil, menampilkan, atau mengintegrasikan informasi tentang buku ke dalam aplikasi atau sistem mereka secara efisien dan fleksibel. beberapa fungsi utama REST API dalam konteks koleksi buku perpustakaan: penelusuran data buku, akses metadata buku, integrasi antarplatform, akses real-time, dan penyediaan statistik

B. FITUR

Fitur yang ada pada REST API yaitu CRUD

- **Menambahkan Data Pengusul (Create)**

Fitur ini memungkinkan penambahan data pengusul baru ke dalam database perpustakaan. Data yang dimasukkan, seperti nama, alamat, atau informasi lainnya, disimpan dalam format JSON. Informasi ini kemudian dapat dimanfaatkan oleh pihak Perpustakaan Indramayu untuk berbagai kebutuhan administratif dan operasional.

- **Melihat Data Pengusul (Read)**

Fitur ini digunakan untuk mengambil dan menampilkan informasi pengusul yang telah tersimpan dalam database. Data tersebut dikemas dalam format JSON, yang nantinya dapat di-decode untuk ditampilkan atau diproses lebih lanjut oleh sistem.

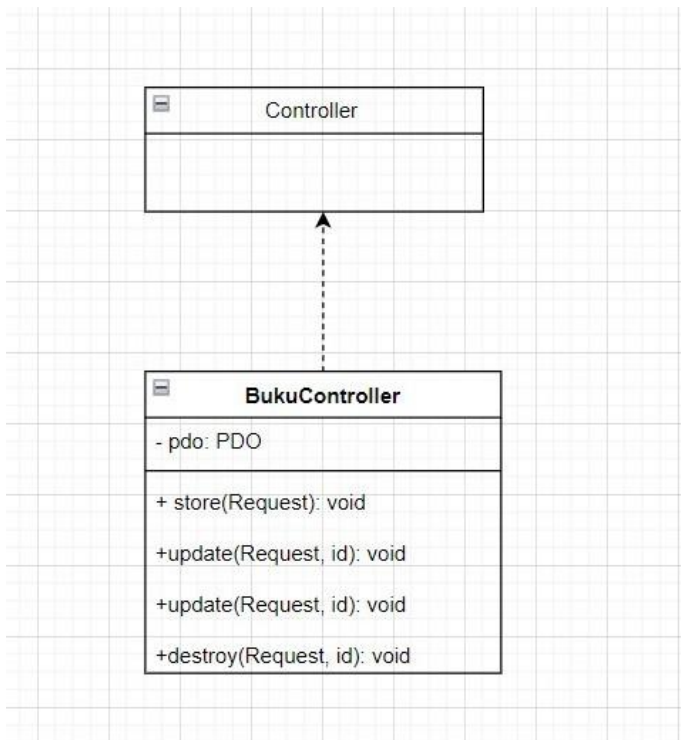
- **Memperbarui Data Pengusul (Update)**

Dengan fitur ini, informasi pengusul yang sudah ada dalam sistem dapat diperbarui sesuai kebutuhan. Setiap perubahan pada data akan langsung tercermin dalam database dan disimpan kembali dalam format JSON yang terstruktur.

- **Menghapus Data Pengusul (Delete)**

Fitur ini memungkinkan penghapusan data pengusul berdasarkan ID unik yang dimiliki setiap entri. Operasi ini memastikan data yang tidak lagi relevan atau diperlukan dapat dikeluarkan dari sistem dengan mudah.

C. Rancangan class diagram



D.Langkah Pengerjaan

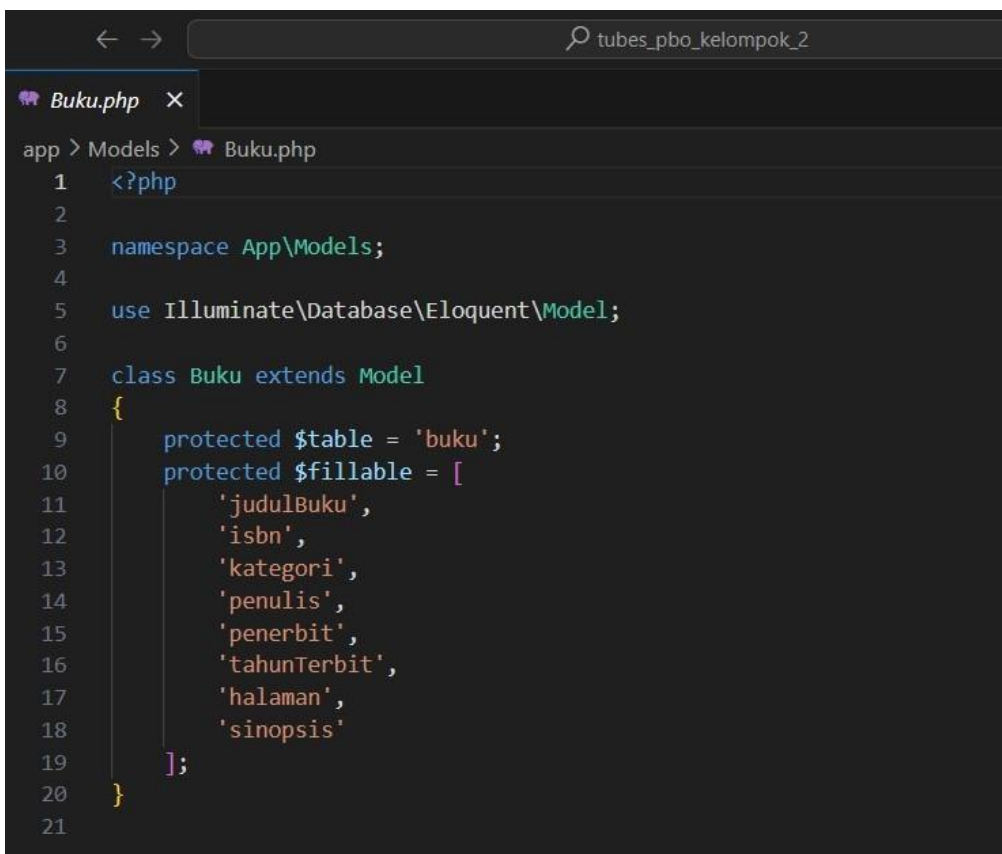
1. buat file model Pengusul dan migrasi dalam satu perintah “*php artisan make:model Buku-M*”
2. Konfigurasi migrasi nya

```
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12      public function up(): void
13      {
14          Schema::create('buku', function (Blueprint $table) {
15              $table->id();
16              $table->string('judulBuku');
17              $table->string('isbn')->nullable();
18              $table->string('kategori');
19              $table->string('penulis');
20              $table->string('penerbit');
21              $table->string('tahunTerbit');
22              $table->text('halaman');
23              $table->text('sinopsis');
24              $table->timestamps();
25          });
26      }
27  }
```

3. Lalu masukkan perintah “*php artisan migrate*” untuk membuat database dan tabelnya

4. Pada bagian model yang ditampilkan, kelas `Buku` merupakan representasi model dari tabel `buku` dalam basis data. Model ini berada dalam namespace `App\Models` dan memperluas kelas `Model` dari Laravel's Eloquent ORM. Dengan menggunakan Eloquent, model `Buku` berfungsi sebagai penghubung antara tabel database `buku` dan logika aplikasi, memungkinkan manipulasi data menggunakan pendekatan berbasis objek.

Properti `protected $table = 'buku';` mendefinisikan bahwa model ini terhubung dengan tabel `buku` di basis data. Selain itu, properti `$fillable` mendeklarasikan kolom-kolom yang dapat diisi secara massal (mass assignment) ketika melakukan operasi seperti `create` atau `update`. Dalam hal ini, kolom-kolom yang dapat diisi meliputi `judulBuku`, `isbn`, `kategori`, `penulis`, `penerbit`, `tahunTerbit`, `halaman`, dan `sinopsis`.

A screenshot of a code editor window titled 'tubes_pbo_kelompok_2'. The editor shows a file named 'Buku.php' with the following PHP code:

```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6
7 class Buku extends Model
8 {
9     protected $table = 'buku';
10    protected $fillable = [
11        'judulBuku',
12        'isbn',
13        'kategori',
14        'penulis',
15        'penerbit',
16        'tahunTerbit',
17        'halaman',
18        'sinopsis'
19    ];
20 }
21
```

5. Membuat API Resources

dapat menjalankan perintah `php artisan make:resource PostResource`. Perintah ini bertujuan untuk membuat sebuah Resource yang digunakan untuk mengubah data dari model menjadi format JSON yang lebih terstruktur, mudah dibaca, dan efisien. Dengan menggunakan Resource, Anda dapat mengatur cara data model ditampilkan dalam respons API tanpa harus memodifikasi langsung modelnya.

```

7
8  class PostResource extends JsonResource
9  {
10     //define properti
11     public $status;
12     public $message;
13     public $resource;
14

```

Untuk membuat method `__construct` dengan tiga parameter yaitu `$status`, `$message`, dan `$resource`, Anda dapat menggunakannya untuk menginisialisasi nilai properti `$status`, `$message`, dan `$resource` pada kelas `Resource`. Method ini akan dipanggil secara otomatis ketika sebuah instance dari `Resource` dibuat, dengan nilai-nilai yang diberikan melalui parameter tersebut.

Parameter `$status` digunakan untuk menyimpan status dari respons, seperti "success" atau "error". Parameter `$message` berfungsi untuk menyimpan pesan yang ingin disampaikan dalam respons API, sementara `$resource` digunakan untuk menampung data utama yang akan dikirimkan ke klien.

```

/**
 * __construct
 *
 * @param mixed $status
 * @param mixed $message
 * @param mixed $resource
 * @return void
 */
public function __construct($status, $message, $resource)
{
    parent::__construct($resource);
    $this->status = $status;
    $this->message = $message;
}

```

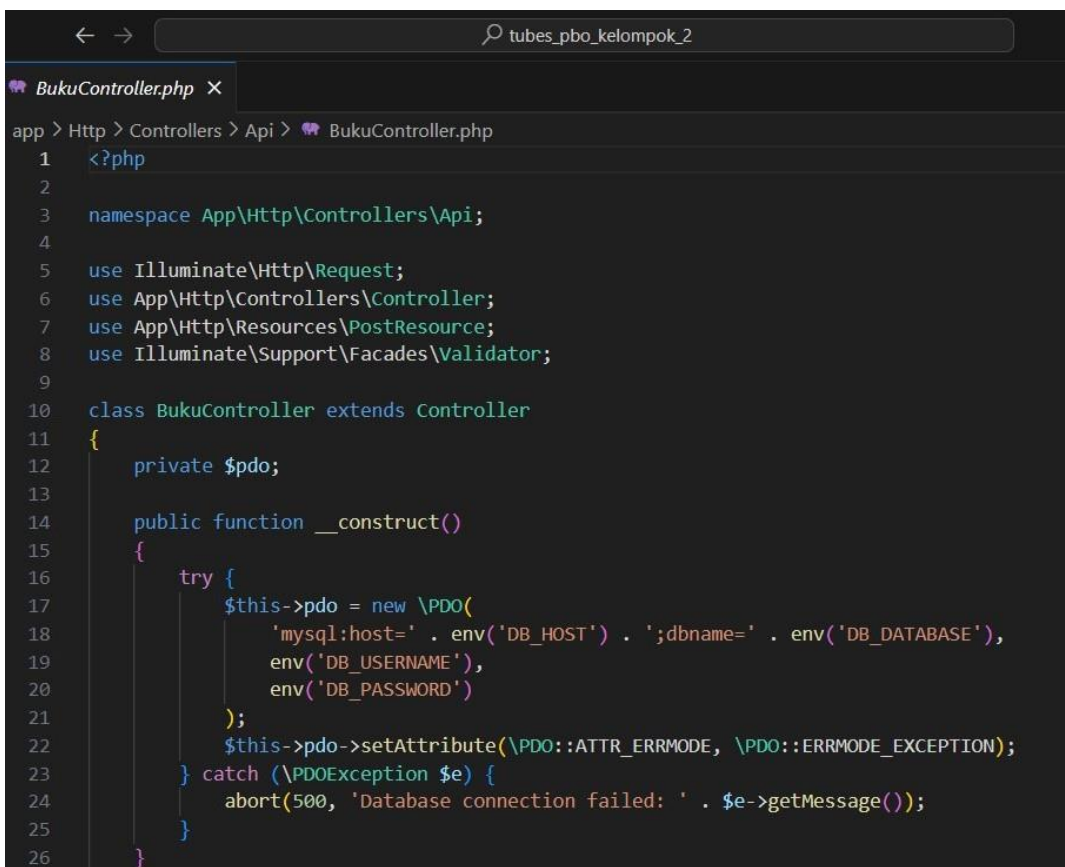
Lalu pada method `toArray()` mengembalikan **status**, **message**, dan **resource** dalam format JSON

```

29
30     /**
31     * toArray
32     *
33     * @param mixed $request
34     * @return array
35     */
36     public function toArray(Request $request): array
37     {
38         return [
39             'success' => $this->status,
40             'message' => $this->message,
41             'data' => $this->resource
42         ];
43     }
44 }

```

6. Kelas ini merupakan turunan dari kelas Controller dan digunakan untuk menangani logika yang berkaitan dengan pengelolaan data buku melalui API. Salah satu fitur yang diimplementasikan adalah koneksi langsung ke basis data menggunakan PDO (PHP Data Objects).



```

← → tubes_pbo_kelompok_2
BukuController.php ×
app > Http > Controllers > Api > BukuController.php
1 <?php
2
3 namespace App\Http\Controllers\Api;
4
5 use Illuminate\Http\Request;
6 use App\Http\Controllers\Controller;
7 use App\Http\Resources\PostResource;
8 use Illuminate\Support\Facades\Validator;
9
10 class BukuController extends Controller
11 {
12     private $pdo;
13
14     public function __construct()
15     {
16         try {
17             $this->pdo = new PDO(
18                 'mysql:host=' . env('DB_HOST') . ';dbname=' . env('DB_DATABASE'),
19                 env('DB_USERNAME'),
20                 env('DB_PASSWORD')
21             );
22             $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
23         } catch (\PDOException $e) {
24             abort(500, 'Database connection failed: ' . $e->getMessage());
25         }
26     }

```

Metode store() ditambahkan untuk menangani penambahan data baru ke dalam tabel pengusul. Dalam metode ini, data yang diterima dari permintaan (request) divalidasi terlebih dahulu untuk memastikan data yang dimasukkan sesuai dengan aturan yang telah ditentukan. Setelah data berhasil divalidasi, blok try menjalankan query SQL INSERT untuk

menambahkan data baru ke tabel pengusul. Data yang diisi berasal dari inputan pengguna melalui objek \$request, kemudian dieksekusi untuk menyimpan data ke dalam basis data.

```
public function store(Request $request)

    $validator = Validator::make($request->all(), [
        'judulBuku' => 'required',
        'isbn' => 'nullable',
        'kategori' => 'required',
        'penulis' => 'required',
        'penerbit' => 'required',
        'tahunTerbit' => 'required',
        'halaman' => 'required',
        'sinopsis' => 'required',
    ]);

    if ($validator->fails()) {
        return response()->json($validator->errors(), 422);
    }

    try {
        $stmt = $this->pdo->prepare("
            INSERT INTO buku (judulBuku, isbn, kategori, penulis, penerbit, tahunTerbit, halaman, sinopsis, created_at, updated_at)
            VALUES (:judulBuku, :isbn, :kategori, :penulis, :penerbit, :tahunTerbit, :halaman, :sinopsis, NOW(), NOW())
        ");
```

```
        $stmt->execute([
            ':judulBuku' => $request->judulBuku,
            ':isbn' => $request->isbn,
            ':kategori' => $request->kategori,
            ':penulis' => $request->penulis,
            ':penerbit' => $request->penerbit,
            ':tahunTerbit' => $request->tahunTerbit,
            ':halaman' => $request->halaman,
            ':sinopsis' => $request->sinopsis,
        ]);

        $id = $this->pdo->lastInsertId();
        $buku = $this->fetchBukuById($id);

        return response()->json(new PostResource(true, 'Data Buku Berhasil Ditambahkan!', $buku), 201);
    } catch (\PDOException $e) {
        return response()->json(['error' => $e->getMessage()], 500);
    }
}
```

7. Mengupdate pengusul (fitur Update REST API)

Metode update() digunakan untuk memperbarui data pengusul berdasarkan ID yang diberikan. Seperti pada metode store(), data yang diterima melalui objek Request divalidasi terlebih dahulu untuk memastikan kesesuaiannya dengan aturan yang telah ditetapkan. Setelah validasi berhasil, query SQL UPDATE disiapkan untuk memperbarui data yang sesuai dengan ID yang dimasukkan. Query ini kemudian dieksekusi menggunakan data yang diperoleh dari \$request.


```

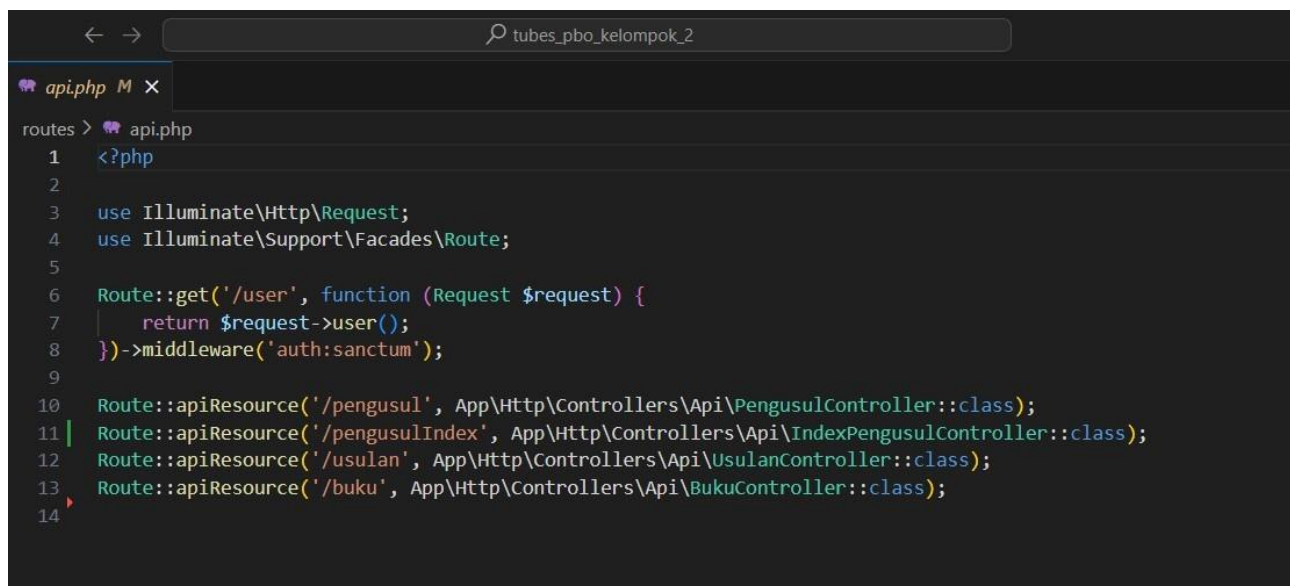
/**
 * Remove the specified resource from storage.
 */
public function destroy($id)
{
    try {
        $stmt = $this->pdo->prepare("DELETE FROM buku WHERE id = :id");
        $stmt->bindParam(':id', $id, \PDO::PARAM_INT);
        $stmt->execute();

        return response()->json(new PostResource(true, 'Data Buku Berhasil Dihapus!', null), 200);
    } catch (\PDOException $e) {
        return response()->json(['error' => $e->getMessage()], 500);
    }
}

```

8. Route API

Pada Laravel 11, untuk membuat route API, langkah awal adalah menjalankan perintah `php artisan install:api`. Perintah ini akan menginstal library API Sanctum yang digunakan untuk mengelola autentikasi token bagi pengguna API. Sanctum memberikan pendekatan yang sederhana dan ringan untuk melindungi API menggunakan token.



```

<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

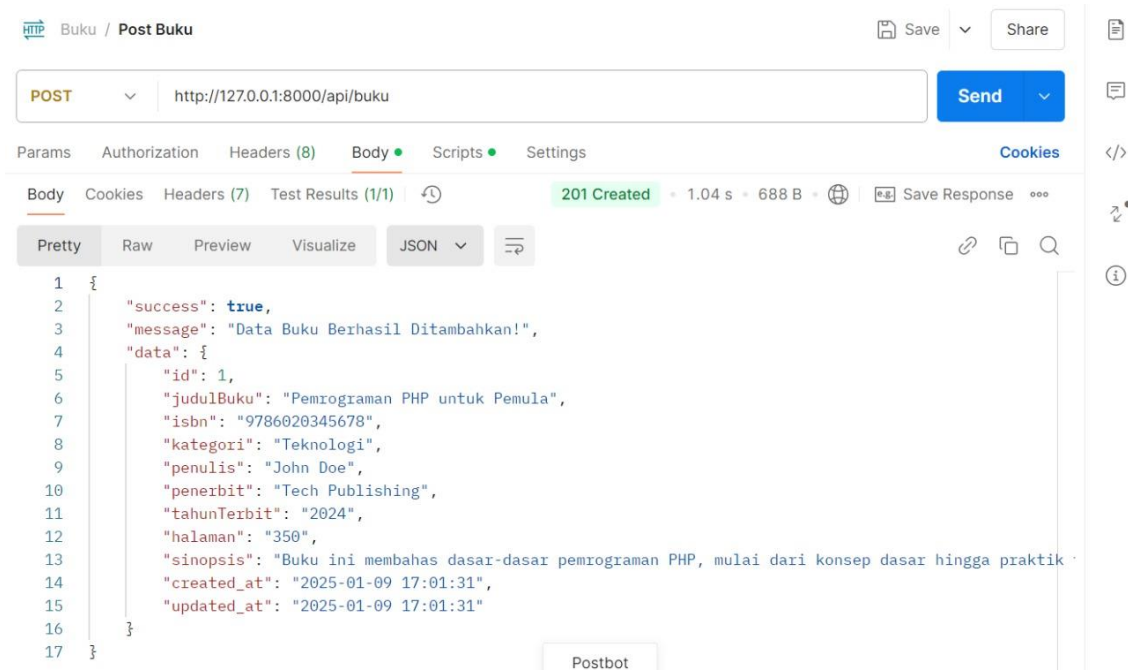
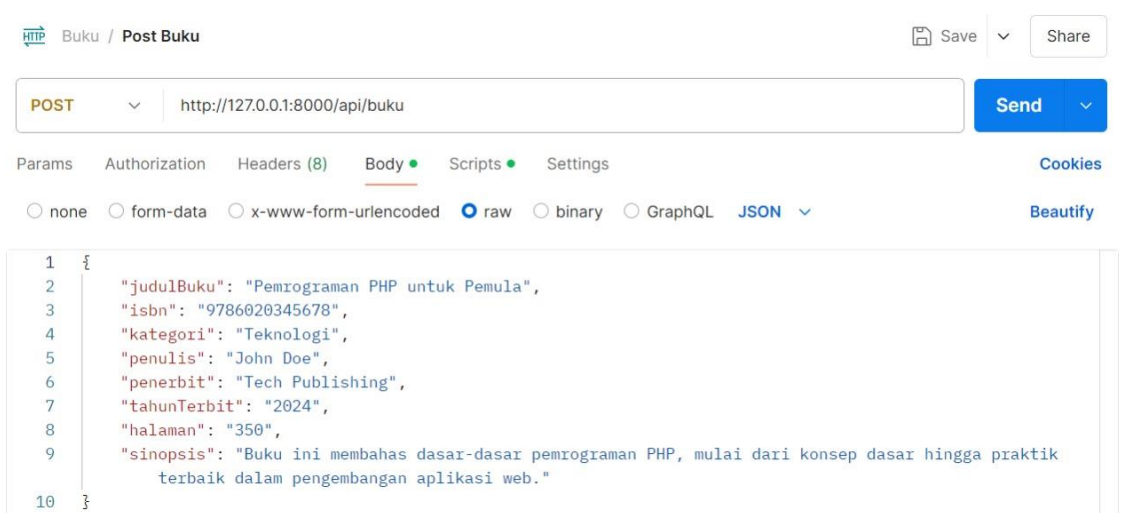
Route::apiResource('/pengusul', App\Http\Controllers\Api\PengusulController::class);
Route::apiResource('/pengusulIndex', App\Http\Controllers\Api\IndexPengusulController::class);
Route::apiResource('/usulan', App\Http\Controllers\Api\UsulanController::class);
Route::apiResource('/buku', App\Http\Controllers\Api\BukuController::class);

```

9. E. Uji coba REST API

1. Post Buku

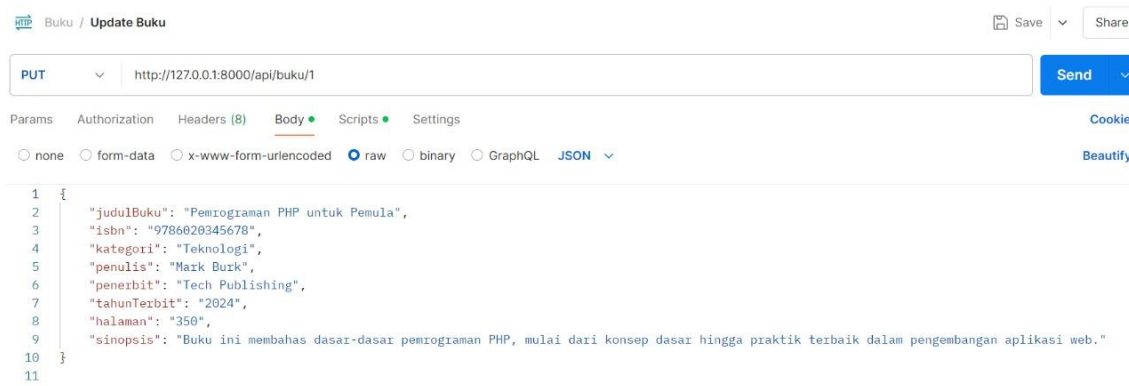
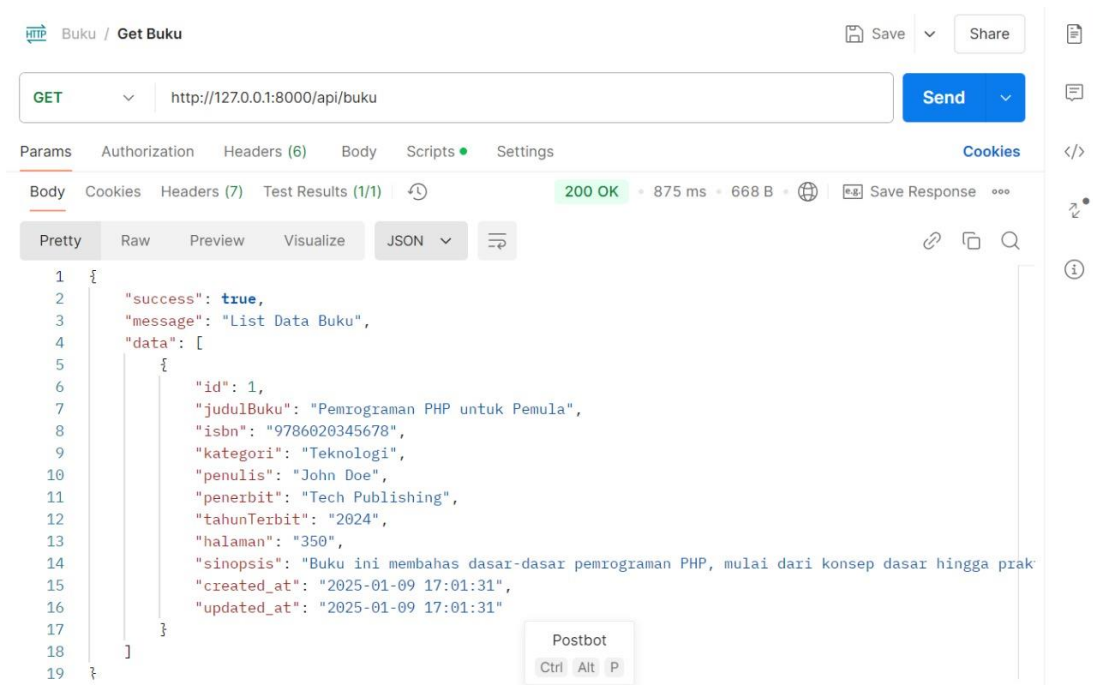
Uji coba



2. Getbuku

asil respons dari API menggunakan metode GET pada endpoint

http://127.0.0.1:8000/api/buku. Respons API dikembalikan dalam format JSON dengan status sukses (true) dan pesan "List Data Buku." Data yang ditampilkan berupa daftar buku, termasuk informasi seperti ID, judul buku, ISBN, kategori, penulis, penerbit, tahun terbit, jumlah halaman, sinopsis, serta tanggal pembuatan (created_at) dan pembaruan (updated_at). Respons ini menunjukkan bahwa API bekerja dengan baik untuk menampilkan data buku dari server.



3.UpdateBuku

pengujian API menggunakan metode `PUT` pada endpoint `http://127.0.0.1:8000/api/buku/{id}` untuk memperbarui data buku berdasarkan ID. Data yang dikirim berupa JSON berisi atribut seperti `judulBuku`, `isbn`, `kategori`, `penulis`, `penerbit`, `tahunTerbit`, `halaman`, dan `sinopsis`. Permintaan ini digunakan untuk mengganti data lama dengan data baru yang dikirimkan melalui body request. Hal ini menunjukkan API mendukung pembaruan data sesuai dengan standar REST API.

HTTP / Buku / Update Buku

PUT http://127.0.0.1:8000/api/buku/1

Send

Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "judulBuku": "Pemrograman PHP untuk Pemula",
3   "isbn": "9786020345678",
4   "kategori": "Teknologi",
5   "penulis": "Mark Burk",
6   "penerbit": "Tech Publishing",
7   "tahunTerbit": "2024",
8   "halaman": "350",
9   "sinopsis": "Buku ini membahas dasar-dasar pemrograman PHP, mulai dari konsep dasar hingga praktik terbaik dalam pengembangan aplikasi web."
10 }
11
```

HTTP / Buku / Update Buku

PUT http://127.0.0.1:8000/api/buku/1

Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Body Cookies Headers (7) Test Results (1/1) 200 OK • 819 ms • 683 B Save Response ***

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": true,
3   "message": "Data Buku Berhasil Diperbarui!",
4   "data": {
5     "id": 1,
6     "judulBuku": "Pemrograman PHP untuk Pemula",
7     "isbn": "9786020345678",
8     "kategori": "Teknologi",
9     "penulis": "Mark Burk",
10    "penerbit": "Tech Publishing",
11    "tahunTerbit": "2024",
12    "halaman": "350",
13    "sinopsis": "Buku ini membahas dasar-dasar pemrograman PHP, mulai dari konsep dasar hingga praktik terbaik dalam pengembangan aplikasi web.",
14    "created_at": "2025-01-09 17:01:31",
15    "updated_at": "2025-01-09 17:03:35"
16  }
17 }
```

Postbot

4.DeleteBuku

HTTP / Buku / Delete data

DELETE http://127.0.0.1:8000/api/buku/1

Send

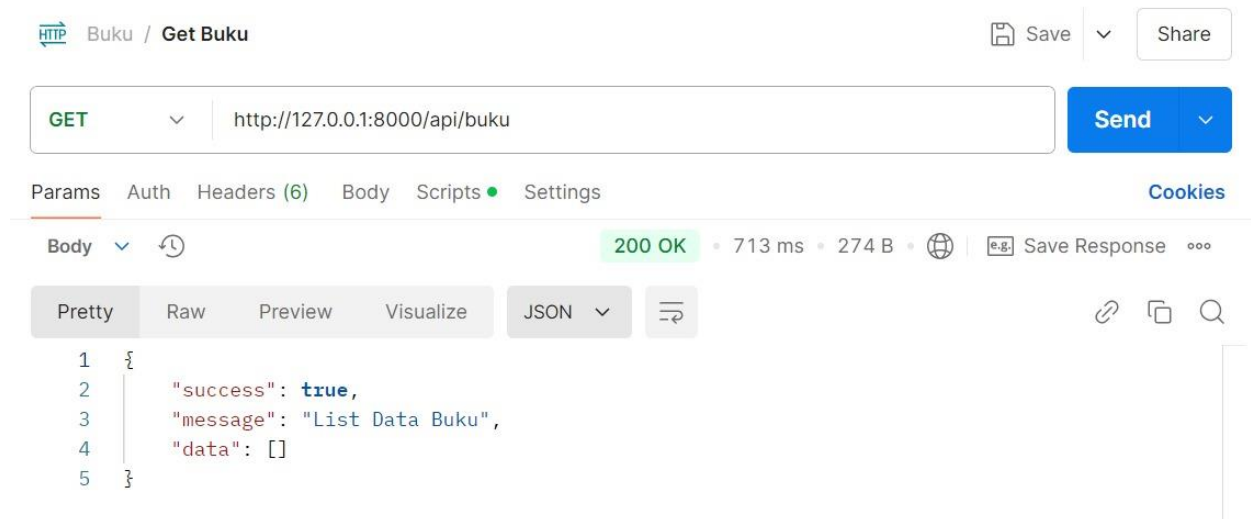
Params Auth Headers (6) Body Scripts Settings Cookies

Body 200 OK • 958 ms • 289 B Save Response ***

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": true,
3   "message": "Data Buku Berhasil Dihapus!",
4   "data": null
5 }
```

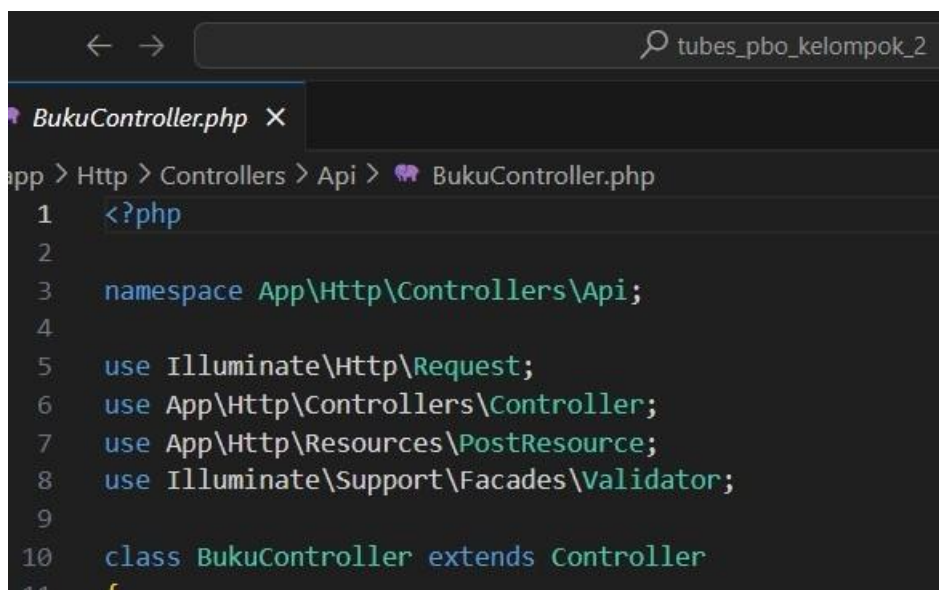
Cek data buku yang sudah di delete, di get buku



F. Materi PBO

1. Class

Class merupakan salah satu konsep dasar dalam pemrograman berorientasi objek. Class berfungsi sebagai cetak biru atau template yang digunakan untuk menciptakan objek. Dalam sebuah class, didefinisikan properti (atribut) dan perilaku (metode) yang akan dimiliki oleh objek yang dibuat berdasarkan class tersebut.



2. Property

Property adalah atribut atau variabel yang terdapat dalam sebuah class, digunakan untuk menyimpan data atau karakteristik yang dimiliki oleh suatu objek. Property menggambarkan informasi atau keadaan yang menjadi bagian dari sebuah objek.

```
{  
    private $pdo;
```

3. Method

Method merupakan fungsi atau prosedur yang terdapat dalam sebuah class dan berfungsi untuk melakukan tindakan atau operasi tertentu pada sebuah objek. Method biasanya bekerja dengan menggunakan property yang ada di dalam class tersebut. Pada contoh di bawah ini, method `__construct` digunakan untuk menginisialisasi koneksi ke database menggunakan PDO.

```
{  
    try {  
        $this->pdo = new \PDO(  
            'mysql:host=' . env('DB_HOST') . ';dbname=' . env('DB_DATABASE'),  
            env('DB_USERNAME'),  
            env('DB_PASSWORD')  
        );  
        $this->pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);  
    } catch (\PDOException $e) {  
        abort(500, 'Database connection failed: ' . $e->getMessage());  
    }  
}
```

4. parameter

Parameter adalah nilai atau argumen yang diberikan ke sebuah method atau constructor untuk digunakan dalam menjalankan suatu proses atau operasi. Dengan adanya parameter, metode menjadi lebih fleksibel karena nilai yang digunakan dapat disesuaikan sesuai dengan kebutuhan yang spesifik.

```
/**  
 * Store a newly created resource in storage.  
 */  
public function store(Request $request)
```

5. Constructor

Constructor pada gambar berfungsi untuk mengatur inisialisasi koneksi ke database dengan memanfaatkan PHP Data Objects (PDO). Hal ini dilakukan untuk memastikan koneksi ke database telah siap digunakan ketika objek dari class tersebut dibuat.

```

public function __construct()
{
    try {
        $this->pdo = new \PDO(
            'mysql:host=' . env('DB_HOST') . ';dbname=' . env('DB_DATABASE'),
            env('DB_USERNAME'),
            env('DB_PASSWORD')
        );
        $this->pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
    } catch (\PDOException $e) {
        abort(500, 'Database connection failed: ' . $e->getMessage());
    }
}

```

6. Object

Object merupakan bentuk nyata dari sebuah class dalam pemrograman berorientasi objek (PBO). Object adalah entitas yang memiliki atribut (properti/data) serta perilaku (method/fungsi) yang telah ditentukan oleh class.

```

class BukuController extends Controller
{
    private $pdo;

    public function __construct()
    {
        try {
            $this->pdo = new \PDO(
                'mysql:host=' . env('DB_HOST') . ';dbname=' . env('DB_DATABASE'),
                env('DB_USERNAME'),
                env('DB_PASSWORD')
            );
            $this->pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
        } catch (\PDOException $e) {
            abort(500, 'Database connection failed: ' . $e->getMessage());
        }
    }
}

```

7. Inheritance

Inheritance adalah proses pewarisan properti dan method dari suatu class ke class lainnya. Pada gambar, controller melakukan pewarisan kepada `PengusulController`, memungkinkan class turunan tersebut untuk menggunakan atau memodifikasi properti dan method yang diwariskan.

```

class BukuController extends Controller
{
    private $pdo;

    public function __construct()
    {
        try {
            $this->pdo = new \PDO(
                'mysql:host=' . env('DB_HOST') . ';dbname=' . env('DB_DATABASE'),
                env('DB_USERNAME'),
                env('DB_PASSWORD')
            );
            $this->pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
        } catch (\PDOException $e) {
            abort(500, 'Database connection failed: ' . $e->getMessage());
        }
    }
}

```

8.Encapsulation

Encapsulation adalah konsep dalam Pemrograman Berorientasi Objek (PBO) yang bertujuan untuk menyembunyikan detail implementasi dari suatu class dan hanya menyediakan interface (antarmuka) yang diperlukan untuk berinteraksi dengan objek tersebut. Dalam hal ini, data dan fungsi yang beroperasi pada data tersebut dibungkus ke dalam satu unit (class), sehingga mempermudah pengelolaan, mengurangi kompleksitas, serta meningkatkan keamanan data. Encapsulation memastikan bahwa hanya bagian-bagian yang relevan dari data dan fungsionalitas objek yang dapat diakses, menjaga kerahasiaan informasi internal dan mencegah manipulasi yang tidak sah.

Properti table dan fillable yang menggunakan `protected` menunjukkan bahwa atribut tersebut hanya dapat diakses oleh anggota class itu sendiri atau turunan dari class tersebut. `Protected` memberikan kontrol akses yang lebih terbatas dibanding `public`, tetapi tetap memungkinkan anak class untuk mengakses properti tersebut. Hal ini membantu menjaga keseimbangan antara aksesibilitas dan keamanan data, karena hanya sebagian tertentu dari data dapat digunakan oleh kelas lain, tetapi tetap terbuka untuk pengembangan dan penggunaan oleh subclass.


```

app > Models > Buku.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Buku extends Model
8  {
9      protected $table = 'buku';
10     protected $fillable = [
11         'judulBuku',
12         'isbn',
13         'kategori',
14         'penulis',
15         'penerbit',
16         'tahunTerbit',
17         'halaman',
18         'sinopsis'
19     ];
20 }

```

9. Abstraction

Abstraction adalah konsep dalam Pemrograman Berorientasi Objek (PBO) yang bertujuan untuk menyembunyikan detail implementasi yang kompleks dan hanya menampilkan fitur atau antarmuka yang relevan kepada pengguna. Dengan kata lain, abstraction memungkinkan kita untuk berinteraksi dengan objek melalui antarmuka yang lebih sederhana tanpa perlu memahami implementasi detail di baliknya. Pengguna hanya perlu mengetahui bagaimana menggunakan fitur-fitur yang tersedia, sementara detail teknis seperti cara kerja internal dari objek tetap tersembunyi dari pandangan mereka.

Bedanya dengan Encapsulation, abstraction lebih berfokus pada menyembunyikan detail kompleks agar pengguna dapat bekerja dengan objek melalui antarmuka yang lebih sederhana. Di sisi lain, encapsulation fokus pada membungkus data dan fungsi ke dalam satu unit (class) untuk mengurangi kompleksitas dan meningkatkan keamanan data. Abstraction memungkinkan pengguna untuk berinteraksi dengan objek menggunakan fitur yang relevan, sedangkan encapsulation bertujuan untuk melindungi data dari manipulasi yang tidak sah dan hanya membolehkan akses pada bagian-bagian tertentu yang diizinkan.

```

abstract class Controller
{
    // You, last week • folder tub
}

```

10. Single Responsibility Principle

Pada class Index buku, hanya menangani satu aksi saja yaitu method index, menampilkan data pengusul.

```

public function index()
{
    try {
        $stmt = $this->pdo->query("SELECT * FROM buku");
        $buku = $stmt->fetchAll(\PDO::FETCH_ASSOC);

        return response()->json(new PostResource(true, 'List Data Buku', $buku), 200);
    } catch (\PDOException $e) {
        return response()->json(['error' => $e->getMessage()], 500);
    }
}

```

11. Try Catch

Di metode **store**, terdapat blok `try` yang digunakan untuk menjalankan kode utama, seperti menyiapkan query SQL, eksekusi query, mengambil ID data baru, mengambil data berdasarkan ID, dan mengembalikan respons JSON. Jika terjadi error, blok `catch` menangkap kesalahan dari PDO, kemudian mengembalikan respons JSON dengan pesan error dan status HTTP 500 (Internal Server Error).

```

public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'judulBuku' => 'required',
        'isbn' => 'nullable',
        'kategori' => 'required',
        'penulis' => 'required',
        'penerbit' => 'required',
        'tahunTerbit' => 'required',
        'halaman' => 'required',
        'sinopsis' => 'required',
    ]);

    if ($validator->fails()) {
        return response()->json($validator->errors(), 422);
    }

    try {
        $stmt = $this->pdo->prepare("
            INSERT INTO buku (judulBuku, isbn, kategori, penulis, penerbit, tahunTerbit, halaman, sinopsis, created_at, updated_at)
            VALUES (:judulBuku, :isbn, :kategori, :penulis, :penerbit, :tahunTerbit, :halaman, :sinopsis, NOW(), NOW())
        ");

        $stmt->execute([
            ':judulBuku' => $request->judulBuku,
            ':isbn' => $request->isbn,
            ':kategori' => $request->kategori,
            ':penulis' => $request->penulis,
            ':penerbit' => $request->penerbit,
            ':tahunTerbit' => $request->tahunTerbit,
            ':halaman' => $request->halaman,
            ':sinopsis' => $request->sinopsis,
        ]);
    }
}

```

```

        $id = $this->pdo->lastInsertId();
        $buku = $this->fetchBukuById($id);

        return response()->json(new PostResource(true, 'Data Buku Berhasil Ditambahkan!', $buku), 201);
    } catch (\PDOException $e) {
        return response()->json(['error' => $e->getMessage()], 500);
    }
}

```

G. Repository

Ini adalah link repo github kelompok 2 D3 Teknik Informatika 2D

https://github.com/vidoonoth/tubes_pbo_kelompok2.git